RÉSUMÉ : INTRODUCTION À LA SOA

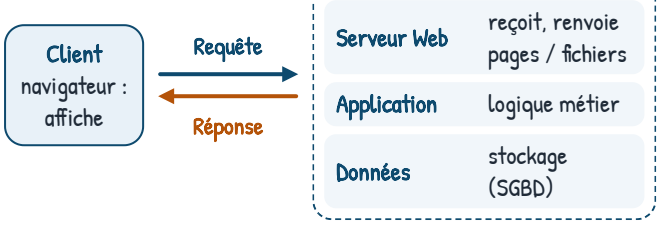
Partie 1/3 — Le Web et ses standards

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 1 · 2026-2027

1 LE FONCTIONNEMENT DU WEB

Principe client-serveur : le **client** envoie une **requête**, le **serveur** la traite et renvoie une **réponse**.



Les 4 couches d'un système Web : client · serveur Web · application · données

2 FORMATS D'ÉCHANGE DE DONNÉES

HTML : langage de balises pour créer et afficher des pages Web.

XML : langage de balises pour structurer et échanger des données.

JSON : format léger d'objets (paires clé / valeur) et de tableaux, très utilisé aujourd'hui.

Ex. : la même donnée dans les deux formats

```xml
<etudiant>
 <nom>Ben Ali</nom>
 <age>19</age>
</etudiant>
```

```json
{
 "nom": "Ben Ali",
 "age": 19
}
```

3 TCP VS HTTP

TCP (Transmission Control Protocol) : couche **transport** ; communication fiable, orientée connexion.

HTTP (HyperText Transfer Protocol) : couche **application** ; s'exécute au-dessus de TCP et régit le transfert des ressources Web entre clients et serveurs.



Segment TCP : le message HTTP voyage à l'intérieur

4 CONTEXTE DE COMMUNICATION

Web centré utilisateur (human-centric) — **B2C** : l'humain initie les requêtes.

Utilisateur → Serveur

Web centré application (application-centric) — **B2B** : des applications d'organisations communiquent entre elles.

Appli A → Appli B

→ Le Web centré application est le terrain des **services Web**.

5 LES MÉTHODES HTTP

GET	POST	PUT	PATCH	DELETE
Récupérer	Créer	Remplacer	Modifier en partie	Supprimer

HEAD : comme GET mais sans corps — la ressource existe-t-elle ? a-t-elle changé ?

OPTIONS : découvrir les méthodes autorisées sur une ressource.

→ Ces méthodes permettent notamment le **CRUD** : Create, Read, Update, Delete.

6 STRUCTURE D'UN MESSAGE HTTP

Requête (3 parties)

Ligne de requête	`GET /etudiants/12 HTTP/1.1`
En-têtes	`Accept: application/json`
Corps (facultatif)	données envoyées (POST, PUT, PATCH)

Réponse (3 parties)

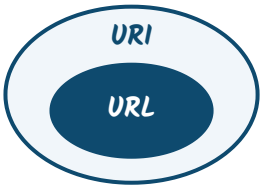
Ligne de statut	`HTTP/1.1 200 OK`
En-têtes	`Content-Type: application/json`
Corps (facultatif)	`{"id":12,"nom":"Ben Ali"}`

7 CODES DE STATUT HTTP

1xx	Information	100 Continue
2xx	Succès	200 OK · 201 Created · 204 No Content
3xx	Redirection	302 Found · 304 Not Modified
4xx	Erreur client	400 Bad Request · 401 Unauthorized · 404 Not Found
5xx	Erreur serveur	500 Internal Server Error

Attention : 400 = requête mal formée · 401 = authentification requise · 404 = ressource introuvable.

8 URI VS URL



URI (Uniform Resource Identifier) : **identifie** une ressource (page, image, vidéo, fichier…).

URL (Uniform Resource Locator) : URI qui indique aussi **comment y accéder** (schéma + adresse).

Ex. : `http://localhost:8080`

Toute URL est un URI, mais tout URI n'est pas une URL.

! À NE PAS CONFONDRE

HTTP (application) s'appuie sur **TCP** (transport) : ils ne sont pas concurrents.

PUT remplace toute la ressource ; **PATCH** n'en modifie qu'une partie.

XML / JSON sont des formats de données ; **HTTP** est le protocole qui les transporte.

RÉSUMÉ : INTRODUCTION À LA SOA

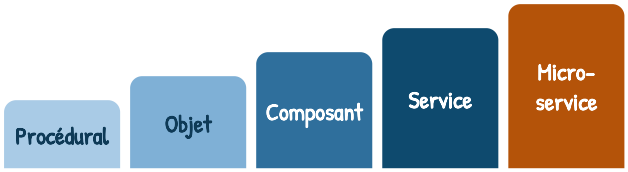
Partie 2/3 — Paradigmes, communication et notion de service

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 1 · 2026-2027

9 LES PARADIGMES LOGICIELS

Paradigme : façon d'organiser le code d'une application — quelles briques on utilise et comment elles s'articulent.



Procédural : procédures appelées en séquence. **Limites** : code « spaghetti », maintenance et réutilisation difficiles.

Objet : des objets qui dialoguent (abstraction, encapsulation, héritage, polymorphisme). **Limites** : couplage fort, réutilisation et maintenance difficiles.

Composant : briques configurables, réutilisables d'une application à l'autre. **Limites** : gestion des dépendances, interopérabilité entre composants hétérogènes.

Service : fonctionnalités exposées par une interface, malgré l'hétérogénéité (langages, plateformes). **Apports** : couplage réduit, réutilisation, abstraction.

Microservice : services autonomes, déployés indépendamment, chacun avec sa propre pile technique. **Apports** : scalabilité, évolution et maintenance facilitées.

10 LE MIDDLEWARE

Middleware : couche intermédiaire qui fait communiquer deux applications en masquant leurs différences (langage, système d'exploitation, plateforme) — comme un traducteur.

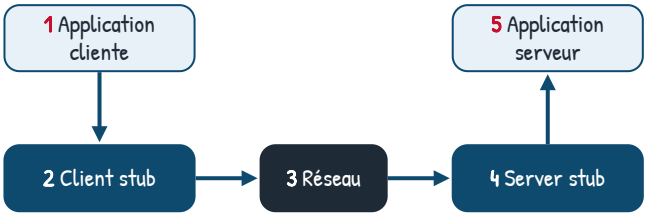


Pourquoi ? **Interopérabilité** : des machines distantes se « parlent » comme si elles étaient voisines. **Hétérogénéité** : fonctionner quels que soient l'OS et le langage de chaque côté.

RPC · générique : appeler une fonction à distance	**RMI** · Java : objets Java distants
CORBA · multi-langages : objets C++, Java…	**DCOM** · Microsoft : propriétaire Windows / .NET

11 RPC (REMOTE PROCEDURE CALL)

RPC : le client appelle une fonction du serveur **exactement comme si elle était locale**.



1 le client appelle · **2** le stub prépare (emballe) l'appel · **3** l'appel voyage sur le réseau · **4** le server stub le réceptionne · **5** le serveur exécute et répond (chemin inverse).

IDL (Interface Definition Language) : le **contrat commun** écrit en premier. Il décrit les fonctions distantes, leurs paramètres et leurs types, indépendamment du langage ; le compilateur IDL en génère les stubs.

12 CORBA

CORBA (Common Object Request Broker Architecture) : le principe du RPC appliqué à des **objets** écrits dans des langages différents.



L'**ORB** localise l'objet distant, transmet l'appel et retourne la réponse : c'est le cœur de CORBA. Un **IDL commun** décrit les interfaces.

	RPC	CORBA
Élément central	Stub / skeleton	ORB
Ce qui communique	Des procédures	Des objets
Multi-langages	Limité	Oui (point fort)
Complexité	Plus simple	Plus riche, plus lourd

13 SERVICE ET SERVICE WEB

Service : unité autonome de fonctionnalités, exécutable indépendamment, qui possède une **interface** pour interagir avec un demandeur.

Local : même machine ou même organisation (services système, applications de bureau). **Distant** : autre machine, sur le Web ou dans le cloud (services Web, cloud).

Service + **Web** = **Service Web**

Un **service Web** met un service à disposition via Internet, pour des applications et systèmes différents. Il doit être :

- **multiplateforme** : client et serveur n'ont pas besoin de la même configuration ;
- **partagé** : plusieurs clients différents l'utilisent en même temps.

RÉSUMÉ : INTRODUCTION À LA SOA

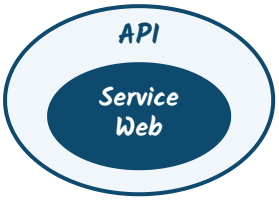
Partie 3/3 — Services Web et architecture SOA

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 1 · 2026-2027

14 SERVICE WEB VS API

Tout service Web est une API ; toute API n'est pas un service Web.



API (Application Programming Interface) : interface qui permet à des composants logiciels de communiquer ; **pas forcément via un réseau.**

Service Web : service accessible par les technologies du Web, pour une communication **machine à machine sur un réseau.**

15 DEUX FAMILLES DE SERVICES WEB

SOAP

Simple Object Access Protocol

- **SOAP** : la communication
- **WSDL** : décrit le contrat
- **UDDI** : publie le service

REST

REpresentational State Transfer

- Utilise directement HTTP
- Une URI identifie chaque ressource
- GET / POST / PUT / DELETE → CRUD

→ Autre famille au programme : **GraphQL** (chapitre 5).

16 INTÉGRATION VS INTEROPÉRABILITÉ

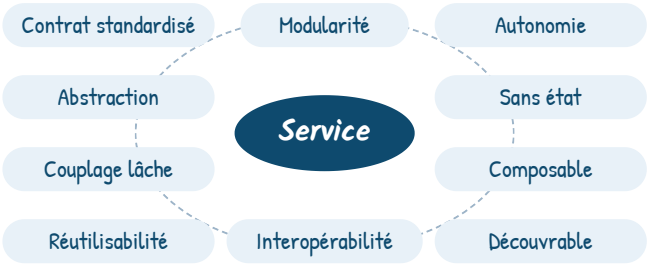
Intégration : du code spécifique relie des systèmes pour qu'ils apparaissent comme **un seul** système.

Interopérabilité : des systèmes indépendants et hétérogènes **coexistent et coopèrent** selon un schéma ouvert d'interconnexion.



2 systèmes indépendants — 1 système « intégré » Sc

17 PRINCIPES D'UN SERVICE WEB



Contrat standardisé (auto-description) : les services d'un même système sont exposés par des contrats aux mêmes règles.

Abstraction : « boîte noire » — seul le contrat est visible, pas la logique ni l'implémentation.

Couplage lâche : dépendance au contrat, pas à l'implémentation ; échanges par messages.

Autonomie : le service contrôle ses ressources et limite ses dépendances externes.

Sans état (stateless) : chaque requête porte tout ce qu'il faut ; aucun état conversationnel entre deux requêtes.

Composable : des services s'orchestrent pour répondre à un besoin complexe.

Modularité : services indépendants, accessibles depuis des environnements hétérogènes.

Interopérabilité : coopérer avec des systèmes hétérogènes (langages, SGBD, architectures différents).

18 AVANTAGES DES SERVICES WEB

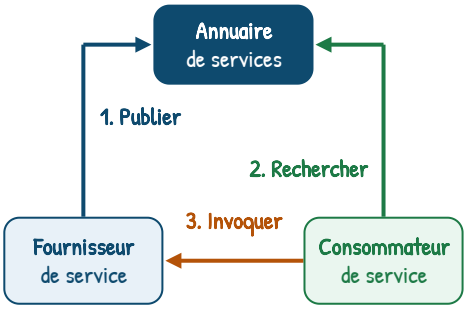
- Technologie adaptée au **B2B**
- Interconnexion de systèmes **hétérogènes**
- Plus ouverts que RPC, DCOM, RMI
- Réutilisables, **couplage lâche**

19 L'ARCHITECTURE SOA

SOA (Service-Oriented Architecture) : architecture qui sépare l'activité métier en une série de **services**, assemblés selon un principe de **couplage lâche** pour construire l'application.

En clair :

- on décompose les capacités métier en services **définis et réutilisables** ;
- chaque service expose un **contrat** qui dit comment l'utiliser ;
- les services sont **assemblés ou orchestrés** pour réaliser



Le triangle de la SOA

Fournisseur : crée le service et publie son interface dans l'annuaire.

Annuaire : rend l'interface et les infos d'accès disponibles à la recherche.

Consommateur : recherche le service dans l'annuaire, puis l'invoque directement.

Apports :

Réutilisation · Développement rapide · Évolution facilitée · Maintenance facilitée · Couplage faible · Standards ouverts

POINTS À RETENIR

Définir avec précision : service, service Web, API, middleware, SOA.

Distinguer : URI / URL, TCP / HTTP, RPC / CORBA, intégration / interopérabilité.

Savoir refaire le triangle SOA et citer les principes d'un service Web.